Q1: 微分程式練習：請參照附圖的微分定理，撰寫 Python 程式以驗證定理 2、3、6、8（i.e. 使用 SymPy 套件）。接著繪製二次方曲線 $f(x) = -10x^2 + 100x + 5$，並求最大值

程式碼如下:

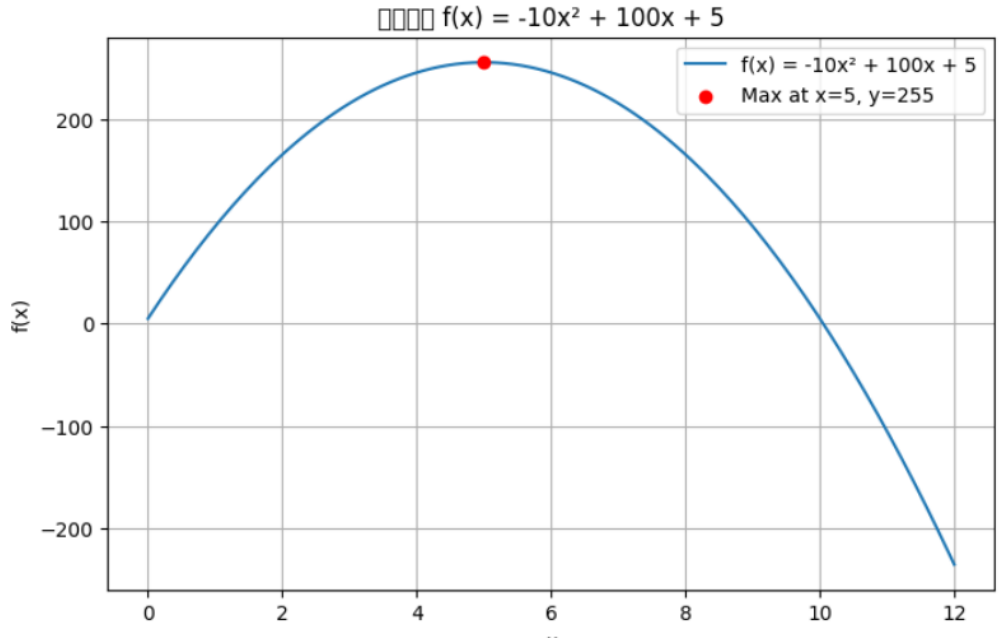
```
import sympy as sp
import matplotlib.pyplot as plt
import numpy as np

x = sp.Symbol('x')                          # 自變數 x
C = sp.Symbol('C', real=True, constant=True)  # 常數 C（固定不變）

# 定理 2：常數函數的導數
# f(x) = C → f'(x) = 0
f2 = C
f2_prime = sp.diff(f2, x)
print("定理 2：f(x)=C → f'(x) =", f2_prime)

# 定理 3：常數倍法則
# f(x) = C*g(x) → f'(x) = C*g'(x)
g = x**3                          # 設 g(x) = x³
f3 = C * g                        # f(x) = C * g(x)
f3_prime = sp.diff(f3, x)     # 對 f(x) 求導
print("定理 3：f(x)=C*g(x) → f'(x) =", f3_prime.simplify())

# 定理 6：乘法法則
# f(x)*g(x) → f'(x)=f'(x)*g(x)+f(x)*g'(x)
f = x**2 + 1
g = sp.sin(x)
lhs = sp.diff(f * g, x)                   # 直接對乘積求導
rhs = sp.diff(f, x) * g + f * sp.diff(g, x)  # 代入乘法法則公式
print("定理 6 驗證結果：", sp.simplify(lhs - rhs) == 0)  # 驗證是否成立
```

```
# 定理 8：鏈鎖律（Chain Rule）
# 若 y = o(i(x))，則 y' = o'(i(x)) * i'(x)
i_expr = x**2 + 1              # 內層函數 i(x)
o_expr = sp.sin(i_expr)        # 外層函數 o(i(x))

# 使用中介符號 u 代表內層函數，以便套用鏈鎖律
u = sp.Symbol('u')
o_u = sp.sin(u)

# 鏈鎖律左右兩邊求導後比較
lhs_chain = sp.diff(o_expr, x)  # 直接對 o(i(x))求導
rhs_chain = sp.diff(o_u, u).subs(u, i_expr) *
sp.diff(i_expr, x)  # 代入公式 o'(i(x))*i'(x)
print("定理 8 鏈鎖律驗證結果：", sp.simplify(lhs_chain -
rhs_chain) == 0)

# 利用導數找最大值
# f(x) = -10x² + 100x + 5
f_curve = -10 * x**2 + 100 * x + 5
f_prime = sp.diff(f_curve, x)          # 求導
x_max = sp.solve(sp.Eq(f_prime, 0), x)[0]  # 解 f'(x)=0 找
出臨界點
y_max = f_curve.subs(x, x_max)         # 將 x_max 代入原函數
得到最大值

print(f"\n 函數 f(x) = -10x^2 + 100x + 5")
print(f"導數 f'(x) = {f_prime}")
print(f"最大值出現在 x = {x_max}, f(x) = {y_max}")

# 繪圖
x_vals = np.linspace(0, 12, 200)                    # x 範
圍 0-12
f_vals = [-10*t**2 + 100*t + 5 for t in x_vals]       # 計
算每個 x 對應的 f(x)

plt.figure(figsize=(8,5))
```

```
plt.plot(x_vals, f_vals, label='f(x) = -10x² + 100x + 5')      # 畫出曲線
plt.scatter([x_max], [y_max], color='red', zorder=5, label=f'Max at x={x_max}, y={y_max}') # 標出最大點
plt.title('二次函數 f(x) = -10x² + 100x + 5')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend()
plt.grid(True)
plt.show()
```

程式碼執行結果:

```
定理2：f(x)=C → f'(x) = 0
定理3：f(x)=C*g(x) → f'(x) = 3*C*x**2
定理6 驗證結果： True
定理8 鏈鎖律驗證結果： True

函數 f(x) = -10x^2 + 100x + 5
導數 f'(x) = 100 - 20*x
最大值出現在 x = 5, f(x) = 255
```



Q2: **二元分類練習：請參照下面連結網頁的說明，試以「Adult」數據集建立多層感知器模型，來預測（識別）年收入（i.e. income 欄位）是否超過$50K。**

https://archive.ics.uci.edu/dataset/2/adult

程式碼如下:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder,
StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score,
classification_report, confusion_matrix


# 載入資料集
columns = [
    "age", "workclass", "fnlwgt", "education", "education-
num",
    "marital-status", "occupation", "relationship",
"race", "sex",
    "capital-gain", "capital-loss", "hours-per-week",
"native-country", "income"
]

train_path = "/content/drive/MyDrive/Adult/adult.data"  #
訓練資料
test_path = "/content/drive/MyDrive/Adult/adult.test"   #
測試資料

train_df = pd.read_csv(train_path, names=columns,
sep=r'\s*,\s*', engine="python")
test_df = pd.read_csv(test_path, names=columns,
sep=r'\s*,\s*', skiprows=1, engine="python")

# 測試資料的 income 標籤多了一個 "."（例如 ">50K."），需移除
```

```
test_df["income"] = test_df["income"].apply(lambda x:
x.replace(".", ""))

# 將 "?" 視為缺失值並刪除整列
train_df = train_df.replace("?", pd.NA).dropna()
test_df = test_df.replace("?", pd.NA).dropna()

# 特徵與標籤分離
X_train = train_df.drop("income", axis=1)
y_train = train_df["income"]
X_test = test_df.drop("income", axis=1)
y_test = test_df["income"]

# 特徵工程（數值標準化 + 類別編碼）
# 數值特徵欄位
numeric_features = ["age", "fnlwgt", "education-num",
"capital-gain", "capital-loss", "hours-per-week"]

# 類別特徵欄位（排除數值欄位）
categorical_features = [c for c in X_train.columns if c
not in numeric_features]

# ColumnTransformer 用於同時處理不同型態欄位
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(),
numeric_features),                    # 對數值特徵進行標準化
(Z-score)
        ("cat", OneHotEncoder(handle_unknown="ignore"),
categorical_features)  # 類別特徵進行 One-Hot 編碼
    ]
)

# 建立 MLP(多層感知器)模型
mlp = MLPClassifier(
    hidden_layer_sizes=(64, 32),  # 兩層隱藏層：第一層 64 個神
經元，第二層 32 個
    activation='relu',            # 使用 ReLU 激活函數
```

```
    solver='adam',                  # 使用 Adam 最佳化器
    max_iter=300,                   # 最多訓練 300 次迭代
    random_state=42                 # 設定隨機種子以確保結果可重現
)

# 使用 Pipeline 串接「前處理」與「分類器」
model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", mlp)
])

# 模型訓練
model.fit(X_train, y_train)

# 模型評估
y_pred = model.predict(X_test)

print("\n=== 模型評估 ===")
print(f"準確率 (Accuracy): {accuracy_score(y_test,
y_pred):.4f}")   # 模型整體準確率
print("\n分類報告 (Classification Report):\n")
print(classification_report(y_test,
y_pred))              # 各類別 Precision / Recall / F1-score
print("\n混淆矩陣 (Confusion Matrix):\n")
print(confusion_matrix(y_test, y_pred))                # 顯
示預測 vs 實際結果的分佈
```

程式碼執行結果:

```
=== 模型評估 ===
準確率 (Accuracy): 0.8240

分類報告 (Classification Report):

              precision    recall  f1-score   support

       <=50K       0.87      0.90      0.89     11360
        >50K       0.66      0.58      0.62      3700

    accuracy                           0.82     15060
   macro avg       0.77      0.74      0.75     15060
weighted avg       0.82      0.82      0.82     15060


混淆矩陣 (Confusion Matrix):

[[10275  1085]
 [ 1566  2134]]
```

Q3: 多元分類練習：請參照下面連結網頁的說明，試以「Cover type」數據集建立多層感知器模型，來預測（識別）森林覆蓋（i.e. Cover_Type 欄位）的類型。

https://archive.ics.uci.edu/dataset/31/covertype

程式碼如下:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report,
accuracy_score
```

```
# 讀取資料並設定欄位名稱
col_names = [
    'Elevation', 'Aspect', 'Slope',
    'Horizontal_Distance_To_Hydrology',
'Vertical_Distance_To_Hydrology',
    'Horizontal_Distance_To_Roadways',
    'Hillshade_9am', 'Hillshade_Noon', 'Hillshade_3pm',
    'Horizontal_Distance_To_Fire_Points'
]

# 四種荒野區類別 One-Hot 編碼
col_names += [f'Wilderness_Area{i}' for i in range(1, 5)]

# 40 種土壤類型 One-Hot 編碼
col_names += [f'Soil_Type{i}' for i in range(1, 41)]
col_names += ['Cover_Type']

# 讀取壓縮檔案
df = pd.read_csv("/content/drive/MyDrive/Cover
type/covtype.data.gz",
                 header=None, names=col_names)

print("資料形狀：", df.shape)
print("欄位示例：", df.columns[:10].tolist(), "...")

# 分離特徵與標籤
X = df.drop(columns=['Cover_Type'])  # 特徵（輸入變數）
y = df['Cover_Type']                 # 標籤（輸出類別）

# 拆分訓練與測試資料
# stratify=y →依照類別比例分層抽樣，確保訓練集與測試集比例一致
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 特徵標準化
# 對數值特徵做 Z-score 標準化，使每個特徵均值為 0、標準差為 1
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)  # 根據訓練資料擬合並轉換
X_test = scaler.transform(X_test)        # 使用同樣的參數轉換測試資料

# 建立 MLP 神經網路模型
mlp = MLPClassifier(
    hidden_layer_sizes=(128, 64),  # 兩層隱藏層：128 與 64 個神經元
    activation='relu',             # 使用 ReLU 激活函數
    solver='adam',                 # 使用 Adam 最佳化器
    max_iter=100,                  # 訓練迭代次數上限（可調大）
    random_state=42,               # 固定隨機種子，確保可重現
    verbose=True                   # 顯示訓練過程（損失收斂資訊）
)

# 訓練模型
mlp.fit(X_train, y_train)

# 模型預測與評估
y_pred = mlp.predict(X_test)
acc = accuracy_score(y_test, y_pred)

print(f"\n 模型準確率：{acc:.4f}")
print("\n 分類報告：")
print(classification_report(y_test, y_pred))
```

程式碼執行結果:

```
資料形狀： (581012, 55)
欄位示例： ['Elevation', 'Aspect', 'Slope', 'Horizontal_Distance_To_Hydrology', 'Vertical_Distance_To_Hydrology', 'Horizontal_Distance_To_Roadways', 'Hillshade_9am', 'Hillshade_Noon', 'Hillsh
Iteration 1, loss = 0.56518756
Iteration 2, loss = 0.45512683
Iteration 3, loss = 0.41032760
Iteration 4, loss = 0.38098073
Iteration 5, loss = 0.35994143
Iteration 6, loss = 0.34454996
Iteration 7, loss = 0.33237805
Iteration 8, loss = 0.32176436
Iteration 9, loss = 0.31302273
Iteration 10, loss = 0.30480207
Iteration 11, loss = 0.29846483
Iteration 12, loss = 0.29270466
Iteration 13, loss = 0.28698953
Iteration 14, loss = 0.28184509
Iteration 15, loss = 0.27822143
Iteration 16, loss = 0.27403435
Iteration 17, loss = 0.27021057
Iteration 18, loss = 0.26742651
Iteration 19, loss = 0.26374479
Iteration 20, loss = 0.26118157
Iteration 21, loss = 0.25798707
Iteration 22, loss = 0.25564944
Iteration 23, loss = 0.25368705
Iteration 24, loss = 0.25111810
Iteration 25, loss = 0.24932985
Iteration 26, loss = 0.24765044
```

```
Iteration 90, loss = 0.19729979
Iteration 91, loss = 0.19813826
Iteration 92, loss = 0.19720933
Iteration 93, loss = 0.19736228
Iteration 94, loss = 0.19550813
Iteration 95, loss = 0.19661018
Iteration 96, loss = 0.19523454
Iteration 97, loss = 0.19570110
Iteration 98, loss = 0.19636889
Iteration 99, loss = 0.19458292
Iteration 100, loss = 0.19374380
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perc
  warnings.warn(

模型準確率：0.9121

分類報告：
              precision    recall  f1-score   support

           1       0.93      0.89      0.91     42368
           2       0.91      0.95      0.93     56661
           3       0.84      0.96      0.89      7151
           4       0.92      0.68      0.78       549
           5       0.85      0.70      0.77      1899
           6       0.88      0.70      0.78      3473
           7       0.95      0.88      0.92      4102

    accuracy                           0.91    116203
   macro avg       0.90      0.82      0.85    116203
weighted avg       0.91      0.91      0.91    116203
```

Q4: 迴歸分析練習：請參照下面連結網頁的說明，試以「Bike Sharing」數據集建立多層感知器模型，來輸出（預測）租賃自行車（i.e. cnt 欄位）的總數。

https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset

程式碼如下:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 讀取資料
df =
pd.read_csv("/content/drive/MyDrive/BikeSharing/day.csv")

# 選擇特徵與目標欄位
X = df.drop(["instant", "dteday", "casual", "registered",
"cnt"], axis=1)
y = df["cnt"]

# 資料分割
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# 標準化
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 建立多層感知器模型
mlp = MLPRegressor(hidden_layer_sizes=(100, 50),
                   activation='relu', # ReLU 激活函數
                   solver='adam',
                   max_iter=1000,# 最大迭代次數（允許充分收斂）
                   random_state=42)
```

```
# 模型訓練
mlp.fit(X_train_scaled, y_train)

# 模型預測
y_pred = mlp.predict(X_test_scaled)

# 模型評估
# 均方誤差（MSE）：預測誤差的平方平均值，越小越好
mse = mean_squared_error(y_test, y_pred)
# R²（決定係數）：衡量模型解釋資料變異的能力，越接近 1 越好
r2 = r2_score(y_test, y_pred)

print(f"均方誤差 (MSE): {mse:.2f}")
print(f"決定係數 (R²): {r2:.3f}")
```

程式碼執行結果:

```
均方誤差 (MSE): 660178.74
決定係數 (R²): 0.835
```